

# Malware in Disguise: Crafting Adversarial Executables

Danny Kim

dkim5@oxy.edu

Occidental College

## 1 Introduction

Machine Learning (ML) algorithms have transformed numerous industries by offering advanced methods for analyzing data, predicting outcomes, and automating complex tasks with their performance being often better than humans. In cybersecurity, these technologies have become vital tools for detecting and responding to a wide array of threats. Traditional malware detection methods such as signature-based detection rely on a unique pattern of identifying features that has been predetermined by a security professional. While this method is effective in detecting known malware variants, it fails to detect new or unknown malware and can be resource-intensive for the system.

Due to the shortcomings of traditional malware detection systems, ML based malware detectors have become a popular area of research for detecting new threats. Unlike traditional malware detectors, ML-based malware detectors require less specialized signatures and can generalize malware signatures to detect new malware that previously hasn't been seen before [20]. While ML-based malware detectors show a promising future, they are vulnerable to adversarial attacks.

Adversarial attacks in this context describe an attack method where the adversary intentionally adds perturbations to the input sample with the goal of misclassification by the detector model[9]. The adversarial malware problem is particularly concerning because it represents a fundamental tension between detection accuracy and robustness. While detectors may perform exceptionally well on naturally occurring samples, adversaries can exploit the learned decision boundaries to evade detection through functionality-preserving binary modifications. Understanding how these attacks succeed requires transparency into model decision-making processes. Most machine learning models operate as black boxes where the relationship between input and classification outputs remains opaque. This opacity hinders both defensive research and trust in deployed systems. If the reason for misclassification isn't known, defenders cannot identify which aspect of their detector requires improvement.

This project explores how explainability techniques can bridge the gap between adversarial attack strategies and model transparency. Specifically, we apply SHAP (SHapley

Additive exPlanations) to analyze how simple functionality-preserving perturbations affect the EMBER malware detector, an industry standard LightGBM-based classifier trained on static PE file features. By computing SHAP values before and after applying perturbations, we can quantify which features contribute most strongly to malicious classification and how adversarial modifications shift those contributions. As governments, corporations, and individuals increasingly rely on ML-based security systems, understanding their failure modes becomes a matter of public safety.

Adversarial malware research occupies an ethical gray area as the same techniques that help defenders strengthen systems can be repurposed by attackers to evade them. This dual-use nature demands careful consideration, a topic addressed in section 7. By conducting transparent research into detector vulnerabilities, the project aims to support the development of more robust, trustworthy systems while acknowledging the inherent risks related to dual-use research.

The remainder of this paper is organized as follows. Section 2 provides technical background, section 3 reviews related work, section 4 describes methodology, section 5 presents results, section 6 discusses conclusion, section 7 examines ethical considerations surrounding dual-use technology, and sections 8 and 9 provide replication instructions and code architecture details to support reproducibility.

## 2 Technical Background

### 2.1 Attack Settings

Understanding adversarial malware requires first clarifying the different attack settings. In practice, white-box and black-box scenarios reflect how much an attacker knows about the model, and this knowledge directly influences the strategies available to them.

#### 2.1.1 White Box

In a white box setting, the adversary has complete knowledge of the model, including its internal layout, parameters, training data, and methods[10]. Because the adversary has intricate knowledge of how the detector functions, they can create malicious samples that are designed to highlight

the model's flaws in order to fool the detector [14]. While white-box attacks effective due to their tailored attack methods, these attacks usually don't transfer to different models [8] and are not characteristic of most real-life scenarios.

### 2.1.2 Black box

Black box settings are more realistic and offer a more practical scenario for real-world applications [19]. A black box setting refers to when the adversary only knows the input and output of a certain model. Adversaries can often infer what kind of algorithm and data the model used based on the intended task the model is designed to accomplish [21] but because of the limited knowledge of the targeted model black box settings are much more difficult.

This study aims to utilize the explainability-driven analysis to guide adversarial perturbations. This requires access to the detector model and thus reflects a white box attack setting.

## 2.2 Adversarial Attacks

Adversarial attacks can be categorized into two main types, evasion and poisoning. Evasion attacks describe a type of attack where the adversary deliberately manipulates the input example with different levels of perturbations intending to fool the ML model to misclassify the input [19]. Poisoning attacks on the other hand occur when adversaries gain access to the training data and can add malicious samples to manipulate the trained ML or DL model [21]. This study examines a specific type of evasion attack called input perturbation.

There are several different types of evasion attacks seen in the current cybersecurity landscape today and are utilized depending on the context of the problem. The three main categories defined by Nightfall ()[18] are: Input perturbation attacks, feature-space attacks, and model inversion attacks.

- Input perturbation attacks involve modifying the input data to evade detection and focus on the adding perturbations to the problem space.
- Feature-space attacks, like the name, involve modifying features used by the model to make its predictions, thus usually involve a white-box scenario where the adversary has knowledge of the features the detector model is utilizing.
- Model inversion attacks, on the other hand, focus on gaining some knowledge of the detector model's parameters or architecture. This is done by repeated querying of the model and using the output to infer its parameters and architecture.

There are also multiple different strategies the adversary can use, including gradient-based and optimization-based, again described by Nightfall ()[18].

- Gradient Based: To generate adversarial samples, gradient-based attacks manipulate the input data according to the gradient of the loss function that results in the misclassification.
- Optimization Based: Follow a similar idea to gradient-based attacks, but utilize algorithms such as evolutionary or genetic to identify and exploit vulnerabilities within a model's decision boundaries

As the ultimate goal of this work is to highlight explainability, a gradient-inspired input perturbation attack was performed.

## 2.3 Explainability

Explainability in AI has recently taken a center role in the conversation of mass adoption of ML into society. In the general context, explainability is important in ensuring ethical application of AI, building trust/confidence in the model, and allows researchers to improve models by understanding how a model arrived at a specific decision/output. In the context of adversarial malware detection, explainability is crucial as it assists in the study of both the attack method and model weaknesses. By analyzing the explanations of a failed adversarial malware sample and a successful adversarial malware sample, analysts can understand the logic behind what makes an attack successful and highlight which features or data regions were exploited during the attack. This allows researchers to understand the vulnerabilities of the model (or the effectiveness of an attack) thus allowing them to create stronger, more resilient models.

### 2.3.1 SHapley Additive exPlanations (SHAP)

Because modern ML models are built upon millions of datapoints and samples, they are often hard to explain, becoming a black box model. One method of making the ML models more transparent and understandable is to calculate SHapley Additive exPlanations (SHAP) values. SHAP values can help you see which features are most important for the model and how they affect the outcome. SHAP values stem from Shapley values, a concept in collaborative game theory where the Shapley values represent the fair distribution of a collective payoff among players based on their individual contributions to a coalition [17]. In game theory, each player's Shapley value is calculated by considering all possible coalitions that the player could join and measuring the marginal contribution the player makes to each coalition. This ensures that each player receives credit propor-

tional to their actual contribution to the overall outcome, satisfying properties of fairness, efficiency, and additivity.

SHAP adapts this game-theoretic framework to machine learning by treating features as "players" and the model's prediction as the "payoff." For a given prediction, SHAP computes each feature's contribution by evaluating the feature across all possible combinations of other features. Specifically, SHAP calculates the average marginal contribution of a feature across all possible orderings of features, measuring how much the prediction changes when that feature is added to a subset of features versus removed from it [17]. This produces a unified measure of feature importance that satisfies three desirable properties: local accuracy (the sum of SHAP values equals the model's output minus the base prediction), missingness (features not used in the model have zero SHAP values), and consistency (if a model changes to rely more on a feature, that feature's SHAP value should not decrease) [17]. By providing these additive, theoretically-grounded explanations, SHAP enables practitioners to understand not just which features matter, but precisely how much each feature contributes to pushing a prediction toward or away from a particular class

## 2.4 PE File Structure

Understanding the Portable Executable (PE) file format is fundamental to this project because it defines the structure that both the malware and the detector operate upon. PE files are the standard executable format for Windows operating systems, containing not just the executable code but also metadata, resources, and structural information that the Windows loader uses to prepare the program for execution.

- **DOS Header:** The PE file begins with the MS-DOS header, always starting with the bytes "MZ" (the initials of Mark Zbikowski, one of the MS-DOS architects). This header occupies the first 64 bytes of the file and provides metadata needed for loading the program in a DOS environment. Its presence mainly signals to the system that the file is a valid executable.
- **DOS Stub:** Immediately following the DOS header is the DOS stub, a tiny program embedded for backward compatibility. When run in a DOS environment, it prints the message "This program cannot be run in DOS mode." Historically, this allowed DOS systems to gracefully handle executables intended for Windows.
- **NT Header:** The NT header begins at the offset specified inside the DOS header. It contains three major components: the Signature, the File Header, and the Optional Header.
  - **Signature:** Always "PE\0\0" in hex. It marks the start of the PE-format structures.

- **File Header:** Contains high-level metadata such as the number of sections, the target architecture, and flags describing file characteristics.
- **Optional Header:** Provides information needed by the loader, including the Magic value (PE32 vs PE32+), the program entry point, required memory sizes, and other parameters needed for mapping the file into memory.
- **Data Directories:** At the end of the Optional Header is the Data Directory table, an array of virtual addresses and sizes pointing to important components such as the Import Table, Export Table, Resource Table, Exception Table, Relocations, and more. These directories help the loader locate structured data elsewhere in the file.
- **Section Header Table:** Immediately after the NT headers is the Section Header Table. Each entry describes one section in the PE file, including its name, raw size on disk, virtual size in memory, RVA, and permission flags (read/write/execute).
- **PE Sections:** PE sections store the executable's actual contents—code, data, and resources. Common sections include:
  - **.text:** Contains the program's executable machine code.
  - **.data:** Contains initialized global and static variables.
  - **.rdata:** Typically holds read-only data such as constants, import tables, and RTTI.
  - **.idata:** Sometimes used by compilers for import-related structures.
  - **.rsrc:** Stores resources such as icons, images, manifests, and dialogs.

These sections are the primary regions manipulated in adversarial malware research, since modifying them impacts detection behavior while ensuring functionality of the malware [4] [5](if the added/alterd sections are not run by the PE file).

## 2.5 Gradient Boosting Models

The detector employed in this study is a Light Gradient Boosted Machine (LightGBM), a specific implementation of gradient-boosted decision trees optimized for efficiency and performance on large datasets.

A decision tree is a supervised machine learning model that makes predictions through a series of hierarchical binary decisions[11]. The tree structure consists of a root node containing all training data, internal nodes representing decision points that split data based on feature thresholds, and leaf nodes that output final predictions. During training, the algorithm partitions the data by selecting the

feature that best separate the samples into classes at each node. For example, an internal node might ask "Is header.11 greater than 0.5?" and subsequently splits the samples to the left or right child node based on the answer. During testing, a sample traverses the tree from root to leaf by following the path determined by its feature values, ultimately reaching a leaf node that provides the classification.

While a single decision tree can be trained quickly and interpreted easily, individual trees are considered "weak learners" because they tend to overfit to training data and achieve limited generalization performance. Gradient boosting addresses this limitation by combining multiple weak learners into a single strong learner through sequential training. The process begins by initializing a base prediction. Then, in each iteration, the algorithm calculates the residual errors from the current ensemble's predictions, trains a new decision tree to predict these residuals, and adds the new tree to the ensemble with a learning rate weight. The final prediction is the sum of all trees' outputs. Each subsequent tree focuses on correcting the mistakes of the previous ensemble, gradually reducing prediction error.

LightGBM distinguishes itself from traditional gradient boosting implementations through two key innovations that significantly improve training efficiency. First, it uses leaf-wise tree growth rather than the conventional level-wise approach. Unlike level-wise algorithms that grow all nodes at the same depth simultaneously, LightGBM uses a best-first approach that splits the leaf with the maximum delta loss. This produces deeper, more asymmetric trees that can achieve lower loss with fewer nodes, improving both accuracy and training speed. Second, LightGBM employs Gradient-based One-Side Sampling (GOSS), which intelligently samples training instances by keeping all samples with large gradients (large errors) while randomly sampling instances with small gradients. This reduces computational cost without significantly sacrificing accuracy [11].

## 2.6 Feature Extraction

Feature extraction is the process of transforming raw input data into a simplified set of features or attributes [12]. This process is done to reduce data complexity and to highlight the most relevant information. It plays an important role in reducing computational costs and improving model accuracy by focusing on essential aspects of the data.

The EMBER baseline model [6] extracts a wide range of static PE features, grouped into two categories: parsed PE features and format-agnostic features.

- Parsed PE Features (require parsing the PE structure):
  - General File Info: Basic attributes such as file size, virtual size, number of imports/exports, and whether the file has resources, TLS, relocations, or a digital signature.

- Header Information: Metadata from the COFF and Optional headers: machine type, subsystem, DLL characteristics, version numbers, and memory layout information.
  - Imported Functions: List of DLLs and API calls the binary depends on, extracted from the Import Address Table.
  - Exported Functions: Any functions the binary exposes for other programs to call.
  - Section Information: Per-section properties such as section names (.text, .data, .rdata), raw size, virtual size, entropy, and permission flags.
- Format-Agnostic Features (do not require parsing):
    - Byte Histogram: Counts of each byte value (0–255) across the file, giving a coarse view of content distribution.
    - Byte-Entropy Histogram: A 2D histogram capturing how byte values correlate with local entropy, useful for detecting packed or obfuscated regions.
    - String Information: Statistics about printable strings (counts, average length, character distributions) and indicators such as URLs, file paths, registry keys, or embedded "MZ" headers.

## 3 Related Work

### 3.1 Machine Learning for Malware Detection

[2] released the EMBER dataset to address the lack of a large-scale, legally shareable benchmark for Windows PE malware detection. The dataset contains features extracted from 1.1M binaries: 900k training samples (300k malicious, 300k benign, 300k unlabeled) and 200k test samples (100k malicious, 100k benign). They also published the extraction code and a LightGBM baseline model achieving 99.3% ROC-AUC on the test set. Notably, even without hyperparameter tuning, this baseline outperformed Mal-Conv, highlighting the strength of engineered PE-specific features compared to early end-to-end byte-based models. The EMBER feature set consists of 2,381 dimensions spanning byte histograms, entropy measurements, import/export table statistics, section characteristics, and string metadata, providing a comprehensive static representation of PE file structure.

In our project, the EMBER baseline detector serves as the target model, allowing us to study how ML-based static detectors can be manipulated through adversarial perturbations at the binary level.

|                 | absolute     |                     | positive     |                     | negative     |                     |
|-----------------|--------------|---------------------|--------------|---------------------|--------------|---------------------|
|                 | section only | section and padding | section only | section and padding | section only | section and padding |
| 1 perturbation  | 2.301987     | 2.616097            | 4.0068       | 3.9991              | -1.3964      | -1.4646             |
| 6 perturbation  | 1.759888     | 1.82604             | 3.2098       | 3.828               | -1.2651      | -1.2027             |
| 20 perturbation | 2.054857     | 1.856496            | 3.7026       | 3.3395              | -1.2431      | -1.1919             |
| 50 perturbation | 1.615423     | 1.63322             | 3.5391       | 3.8615              | -1.3997      | -1.4251             |

Table 1: SHAP values across perturbation strategies and intensities

### 3.2 Problem-Space adversarial Attacks

Early adversarial malware research focused on feature-space attacks that manipulated extracted feature vectors directly, limiting practical applicability. More recent work has shifted toward problem-space attacks that modify actual PE binaries while preserving functionality. Kolosnjaji et al. (2018) [15] examined vulnerabilities in MalConv, one of the earliest end-to-end raw-byte CNN architectures for PE malware detection. They proposed a gradient-guided padding attack that computes the gradient of the model with respect to appended bytes, then optimizes those bytes to reduce malicious confidence through iterative gradient descent. They achieved evasion rates exceeding 60% while modifying less than 1% of the file without breaking functionality since the perturbation occurs in the overlay region beyond the PE structure. This work established the foundation for byte-level evasion in problem space rather than feature space, demonstrating that gradient-based optimization could bridge the gap between theoretical feature-space attacks and practical binary manipulation.

Castro, Schmitt, and Rodosek (2019) [7] investigated automated random mutations as a means to evade ML-based detectors and introduced ARMED, a framework for Automatic Random Malware Modification to Evade Detection. Their approach systematically evaluates eight classes of PE modifications including overlay appending, section injection, padding insertion, DOS stub manipulation, and packing. Unlike traditional optimization-based methods, ARMED applies mutations randomly and measures their impact on 11 AV engines and 3 ML-based detectors. They found that even naive random modifications achieved evasion rates of 15-25% against commercial AVs without any targeting or optimization. Their results emphasize that even naive functionality-preserving changes can have outsized effects on static ML models, suggesting that current detectors have not been sufficiently hardened against adversarial manipulation.

Anderson et al. (2018) [3] introduced a reinforcement learning framework for black-box adversarial attacks against static PE malware detectors. Unlike gradient-based approaches requiring model access, their RL agent learns to chain functionality-preserving modifications (sec-

tion renaming, import addition, padding insertion, packing) through interaction with only the detector’s binary output. Training on 50,000 malware samples, they achieved evasion rates of 12-24% across different malware families while requiring no knowledge of the target model’s architecture or features. Critically, they demonstrated that adversarially-generated samples could reduce detection rates on VirusTotal from 54/65 to 26/65 engines (median), indicating cross-evasion properties. They also showed that adversarial re-training reduced subsequent attack success by 33%, providing early evidence for model hardening strategies. However, they noted that approximately 20% of generated samples failed to execute properly due to parser incompatibilities with the LIEF library.

Luca Demetrio et al. (2021) [16] introduced a family of black-box adversarial attacks focused on query-efficiency and preserving the malware’s original behavior. Unlike Anderson et al.’s RL approach, they employed evolutionary algorithms and genetic programming to systematically explore the space of functionality-preserving mutations. Their optimized perturbations included header manipulation, section injection, and slack space padding, operations that preserve executability but alter feature representations. Surprisingly, their study showed that these optimized perturbations transferred effectively to real antivirus engines, with adversarial binaries evading, on average, 12 commercial AV products on VirusTotal. They achieved evasion rates of 37-74% against ML-based detectors using fewer than 100 queries in many cases, demonstrating that small functionality-preserving mutations can lead to significant misclassification even under black-box constraints.

### 3.3 Explainability-Guided Adversarial Attacks

While the attacks described above use optimization (gradient descent, RL, genetic algorithms) to discover effective perturbations, recent work has explored whether explainability techniques can directly identify high-impact modification targets.

Lundberg and Lee (2017) [17] introduced SHAP (SHapley Additive exPlanations), a unified framework for interpreting machine learning predictions based on game-theoretic Shapley values. SHAP assigns each feature an importance value representing its marginal contribution to

the model’s prediction, satisfying three desirable properties: local accuracy (the explanation matches the model output), missingness (absent features receive zero attribution), and consistency (if a feature’s contribution increases across models, its attribution should not decrease). They proved that Shapley values are the unique solution satisfying these properties for additive feature attribution methods.

Lundberg and Lee (2017) [17] introduced multiple estimation methods: Kernel SHAP (model-agnostic using weighted linear regression), Tree SHAP (exact computation for tree-based models), and Deep SHAP (an extension of DeepLIFT for neural networks). For tree-based models like the EMBER LightGBM detector, Tree SHAP provides exact Shapley value computation in polynomial time. While SHAP has been widely adopted for model transparency and debugging, its potential for adversarial attack optimization remained largely unexplored until recently.

Aryal et al. (2024) [6] pioneered the application of explainability techniques to strategize adversarial attacks against malware detectors. They used SHAP to compute feature attribution for MalConv, a CNN-based raw-byte classifier, identifying which byte sequences most strongly influence malware classification decisions. By aggregating byte-level SHAP values across PE structural regions (DOS header, COFF header, Optional Header, sections, overlay), they discovered that the Optional Header exhibited the highest absolute SHAP contribution (mean absolute SHAP = 1.60), correlating with a 75.6% evasion rate when perturbed despite being only 223 bytes on average. In contrast, append attacks (overlay manipulation) showed only 7.7% evasion despite being commonly used in prior work, demonstrating that conventional wisdom about effective attack locations may be suboptimal.

Critically, Aryal et al. (2024) [6] extended this analysis to fine-grained subsection targeting: dividing large sections like .text (average 97KB) into 4KB subsections and targeting those with highest aggregated SHAP values. This increased evasion rates from 17.6% (lowest-SHAP subsection) to 57.4% (highest-SHAP subsection) within .text, a 3.3× improvement. Their findings established a strong positive correlation between absolute SHAP values and evasion efficacy, suggesting that explainability can dramatically enhance attack efficiency by identifying high-impact regions. However, their perturbations were not all functionality-preserving: modifications to the Optional Header, COFF header, and PE signature break file integrity despite achieving high evasion rates. This limits the practical applicability of their findings to real-world adversarial scenarios where malware must remain executable.

## 4 Methods

The goal of this project is to explore how explainability signals relate to a model’s final classification decision. The workflow begins by selecting a malware sample, computing its SHAP values, applying perturbations to generate an adversarial variant, recomputing SHAP values for the modified file, and finally comparing the differences in attribution. This comparison allows us to understand how small structural changes shift model reasoning.

### 4.1 Model Set up

The detector used in this project is the EMBER baseline model, a LightGBM classifier trained on feature embeddings extracted from the EMBER 2018 dataset. The full feature set includes roughly 2,300 dimensions spanning PE metadata, byte histograms, entropy histograms, import/export information, and string statistics.

Feature extraction and model training followed the official EMBER codebase [2], using default parameters to maintain consistency with the widely referenced baseline model in existing literature.

### 4.2 Perturbation Process

This project implements two simple, functionality-preserving perturbation methods adapted from Anderson et al. (2018) [3]:

- **Section Injection:** A new PE section is created with a randomly generated name and random binary content. Each injected section is marked as non-executable and positioned immediately after existing sections. This approach avoids modifying executable code regions, reducing the likelihood of breaking program behavior.
- **Padding:** This method identifies unused space within a randomly selected section and fills it with random bytes. PE files frequently contain slack space due to section alignment rules, meaning the on-disk size often exceeds the size of meaningful content. Padding takes advantage of that gap.

While more sophisticated adversarial techniques exist in current research (such as code-cave loaders, gradient-based optimization in problem space, and genetic search), section injection and padding were intentionally chosen for their clarity. These methods allow the project to cleanly isolate explainability effects without the added complexity of advanced obfuscation techniques. They are also straightforward to implement, minimally disruptive to program behavior, and appropriate given project time constraints.

### 4.3 Adversarial Sample Generation

Adversarial variants were produced using an automated batch-generation script capable of generating one or many perturbed versions of a single malware sample. This process ensures consistent experimentation and reproducibility.

Two perturbation strategies were used:

- Strategy 1: Only section adding modifications utilized
- Strategy 2: Combined approach incorporating both section modifications and padding injection.

Each strategy was repeated 1, 6, 20, and 50 times to ensure a diverse range of perturbations were present in each tested sample.

### 4.4 Evaluation Metrics

SHAP values were computed to identify the features contributing most strongly to malicious classification in the EMBER detector. For each sample, the top 10 features were summarized in three ways: the absolute contribution, the positive contribution, and the negative contribution. The absolute contribution highlights features most influential to the prediction overall; positive SHAP values correspond to features pushing the classification toward malicious, while negative values correspond to features pushing it toward benign.

SHAP was chosen because it provides a well-established, efficient algorithm for tree-based models (shap.TreeExplainer), making it a natural fit for LightGBM. It offers a consistent and interpretable metric to compare pre- and post-perturbation model behavior. Evasion rate was used as a secondary metric to quantify how often the perturbed sample bypassed commercial antivirus engines. Evasion rate is defined as:

$$ER = \frac{\text{number of engines detecting the file as benign}}{\text{total number of engines}}$$

Although the perturbation methods in this project were intentionally simple, they still caused measurable shifts in detection outcomes across commercial antivirus platforms. SHAP provides insight into how perturbations impact the internal EMBER classifier, but does not generalize beyond the specific model. Evasion rate complements SHAP by demonstrating the broader, cross-detector effect of the modifications.

## 5 Results and Discussion

### 5.1 Shap-Based Feature Importance

For the original malware sample, the sum of the top 10 absolute SHAP values was 2.616, composed of 3.999 total

positive contribution (malicious evidence) and -1.465 negative contribution (benign evidence). These values form the baseline against which all perturbation effects were compared.

Our SHAP analysis revealed that data directory features (datadirectories.4 and datadirectories.8) contributed the highest positive impact toward malicious classification. Data directories in PE files specify locations of important tables like import tables, export tables, resource directories, and debug information. High SHAP values for these features suggest the EMBER model learned that certain patterns in data directory counts, sizes, or configurations strongly correlate with malicious behavior (potentially reflecting common malware packing, obfuscation, or resource embedding techniques).

PE header features (header.11, header.53, header.0, header.29) also showed substantial positive contributions. These likely correspond to fields such as TimeDateStamp, characteristics flags, size fields, or subsystem values. Malware often exhibits distinctive header patterns due to automated generation, specific compiler configurations, or intentional header manipulation for evasion purposes [5].

Section properties (section.242, section.3, section.48) contributed moderately to malicious classification. Sections with unusual characteristics such as writable-executable permissions, size discrepancies between virtual and raw sizes, or non-standard section names are common malware indicators that EMBER’s training data captured. Because padding only changes the raw size, it’s understandable that this feature captured the discrepancies between the two. This can be seen in 50 perturbations where header.11, header.53, and header.0 all displayed higher positive values for the padding + section adding attack.

### 5.2 Perturbation Strategy Performance

As seen in Table 1, at 1 perturbation, section-only modifications achieved substantial absolute SHAP reduction (2.616→2.302, 12.0%) while the combined approach showed no improvement. This suggests that adding a single section directly modifies high impact features such as section counts or data directory entities that the detector relies on heavily. The unified approach however showed no improvements suggesting that a single padding injection is not sufficient enough to alter statistical features such as byte histograms or entropy.

At 6 perturbations, both strategies converged in effectiveness (1.760 vs. 1.826), suggesting padding’s cumulative statistical effects begin manifesting while section additions reach diminishing returns.

At 50 perturbations, both strategies plateaued at similar absolute SHAP values (1.615 vs. 1.633), indicating a fundamental limitation of persistence of positive features. De-

spite aggressive perturbations, positive SHAP contributions remained high (3.539-3.862), indicating that core malicious characteristics that impact this malicious sample: likely related to imports, exports, or string artifacts. Because these features remain unmodified, they continue to drive up the malicious score.

A notable non-monotonic trend appears between 6 and 20 perturbations, where the absolute SHAP sum increases (1.7599  $\rightarrow$  2.0549). Examining the SHAP decomposition reveals that negative evidence slightly strengthened (-1.265  $\rightarrow$  -1.234), suggesting perturbations were partially effective, but positive evidence increased significantly (3.2098  $\rightarrow$  3.7026). This behavior reflects the non-linear decision boundaries of gradient-boosted trees where perturbations can simultaneously move a sample across multiple decision thresholds. Gradient boosted trees learn non-linear decision boundaries, thus each perturbation may push the sample across multiple decision thresholds at the same time. In this case, the twenty-perturbation variant likely activated a higher-impact leaf combination than the six- or fifty-perturbation variants.

### 5.3 Limitations

Our explainability-guided approach successfully reduced SHAP-identified high-impact features but achieved only 37-38% absolute SHAP reduction. The persistence of positive features highlights the core limitations of this project of incomplete perturbation coverage. Because padding and section injection targets sections and headers, critical features such as imports/exports and strings remain unmodified thus providing a ceiling that the generated adversarial samples cannot pass without more intentional perturbations. Thus this leads to a natural direction for future project directions where additions of more perturbation methods can be utilized to fulfill the problem space.

### 5.4 Against Commercial Detectors

Table 2 illustrates the impact of incremental perturbation on malware detection outcomes across 72 commercial antivirus engines. The unmodified payload had an evasion rate of 0.431 but after a single perturbation, the number of engines flagging the sample as malicious dropped to 34, increasing evasion rate to 0.528. When six perturbations were applied it resulted in an evasion rate of 0.542.

These results demonstrate a clear increase in evasion rate as additional perturbations are introduced, despite the payload’s core malicious functionality remaining unchanged. Notably, the largest relative gain occurs after the first perturbation, suggesting that even minimal structural or padding-based modifications can significantly disrupt signature-based or feature-sensitive detection mechanisms. Subse-

quent perturbations yield diminishing but still positive returns, indicating a saturation effect where additional perturbations are less effective. Overall, this trend highlights the vulnerability of static malware detectors to cumulative, low-cost perturbations and underscores the importance of robustness-aware evaluation in malware detection research.

## 6 Conclusion

This work demonstrates that SHAP explainability can effectively guide adversarial perturbation strategies against machine learning malware detectors. By identifying and targeting high-impact features (data directories, PE headers, section properties), we achieved substantial SHAP value reductions while preserving malware functionality. However, the non-monotonic perturbation effects and persistence of positive features highlight the complex, non-linear decision boundaries in tree-based detectors and the necessity of comprehensive feature space coverage for successful evasion. These findings reinforce the broader challenge in adversarial malware research of achieving evasion without compromising functionality.

## 7 Ethical Considerations

Dual-use technology refers to tools, methods, or knowledge that hold both beneficial and harmful potential [1]. GPS can guide both civilian navigation and missile systems; knives function as kitchen tools and weapons. In cybersecurity, dual-use concerns arise when research can inform both defensive innovation and offensive misuse. This project sits squarely within that domain: techniques that help analysts understand evasion can also be repurposed by adversaries to develop better evasion strategies.

Biology provides a useful parallel through the framework of Dual Use Research of Concern (DURC), in which an institutional review entity evaluates whether research might enhance the harmful properties of an agent or technique and whether safeguards sufficiently mitigate that risk [13]. While computer security lacks a formal counterpart to DURC, the underlying ethical questions are similar: Does this work unintentionally lower the barrier for attackers? Could its methods or findings be misapplied?

**Potential Risks** Several risks accompany adversarial malware research, even when conducted for defensive purposes:

- **Lowering the barrier to evasion:** Demonstrating how PE structures can be manipulated—particularly when guided by explainability signals—could enable malicious actors to replicate similar perturbations with minimal expertise. Although the techniques implemented in this project are well-known and elemen-



| Sample             | Perturbations | Malicious Detections | Evasion Rate (ER) |
|--------------------|---------------|----------------------|-------------------|
| Unmodified payload | 0             | 41 / 72              | 0.431             |
| Modified payload   | 1             | 34 / 72              | 0.528             |
| Modified payload   | 6             | 33 / 72              | 0.542             |

Table 2: Detection results and evasion rates for payloads with increasing numbers of perturbations.

tary, providing accessible code on a public platform like GitHub may still unintentionally streamline their adoption.

- Exposing model blind spots: Analyzing SHAP values and demonstrating where the EMBER baseline detector is most vulnerable may offer adversaries insight into specific weaknesses or under-protected feature regions.
- Normalizing malware handling: Working with live binaries introduces inherent dangers, including accidental execution or propagation if appropriate containment procedures are not followed.
- Contributing to an arms-race dynamic: Adversarial ML research simultaneously strengthens defenses and pressures attackers to escalate. Even when the intent is defensive, any advancement in evasion techniques can be viewed as fueling a cycle that forces continued escalation by both sides.

It is important to note that while this project’s implementation is public, the techniques used are not novel; the underlying attack methods were already openly available prior to this work. No new evasion capability was introduced, and the contributions primarily involve analyzing explainability signals rather than developing more powerful adversarial strategies.

**Potential Benefits** The defensive value of adversarial malware research is substantial:

- Strengthening classifier robustness: Understanding how perturbations alter model outputs allows defenders to anticipate weaknesses and integrate more resilient architectures or training procedures.
- Improved transparency and accountability: Explainability tools such as SHAP highlight which features drive malicious classifications. This transparency supports principled auditing and debugging of deployed systems.
- Developing proactive defenses: Insights gained from adversarial attempts can inform adversarial training, improved feature engineering, and ensemble detection strategies.
- Identifying limitations of static detection: Demonstrating structural vulnerabilities in PE-based models encourages the field to diversify detection pipelines, invest in multi-modal systems, and acknowledge the limits of static analysis alone.

In short, understanding how detectors fail is essential for building detectors that fail less often.

**Mitigation and Responsible Conduct** Given the dual-use nature of this project, several precautions were taken to minimize risk:

- All malware samples were handled exclusively inside isolated, non-networked virtual machines.
- Only publicly available datasets and detectors were used; malware samples generated were only created using public tools.
- Perturbations were intentionally minimal and did not enhance malware functionality in any way.
- The project’s primary focus is interpretability — not enabling operational attack tooling.
- Results are framed to support defensive improvement, emphasizing detector limitations and opportunities for hardening rather than exploitation.

These measures reflect an intent to ensure that the work remains grounded in scientific inquiry and defensive utility rather than operational misuse.

Adversarial malware research, much like biological DURC, exists at the intersection of innovation and risk. It requires ongoing reflection on intent, impact, and dissemination. This project operates under the assumption that transparent research, careful experimentation, and open discussion of model weaknesses ultimately contribute to a stronger cybersecurity ecosystem. However, acknowledging the dual-use potential of these methods is essential.

## 8 Replication Instructions

This project can be replicated by any computer science student with prior experience using Python and command-line tools. The source code is publicly available on GitHub at <https://github.com/FlyingTurkeySandwich/emberDetector> and can be obtained by cloning the repository using Git. The project is implemented in Python and was developed and tested using Python 3.8. To ensure reproducibility and avoid dependency conflicts, LIEF 0.9.0 should be used. All required Python dependencies should be installed using pip, either via a provided requirements.txt file or by manually installing the necessary libraries, including LightGBM for model training and standard data science libraries such as NumPy and pandas.

The project relies on the EMBER malware benchmark dataset, which is not included in the repository due to its size and licensing constraints. Users must independently download the EMBER dataset from the official Elastic EMBER repository and place it in a local directory accessible to the training scripts. The dataset consists of pre-extracted static features derived from Windows Portable Executable (PE) files and is used directly as model input. Once the dataset path is correctly specified, the main training script can be executed to train a LightGBM-based malware classifier. The training process reads the EMBER feature vectors, fits the model, and outputs a trained classifier that can then be evaluated on held-out test data.

## 9 Code Architecture Overview

The project is organized as a modular static malware analysis pipeline, with clear separation between dataset handling, model training and evaluation, adversarial sample generation, and analysis utilities. At the top level, the repository contains dedicated directories for data storage, feature extraction, model execution, and adversarial experimentation. Raw and processed datasets are stored under the `data/` and `data/ember2018/` directories.

Core EMBER functionality is encapsulated in the `ember/` directory, which handles feature loading and interaction with the EMBER 2018 feature representation. This module acts as the interface between raw dataset artifacts and downstream machine learning components. Model training and inference are implemented in the `scripts/` directory, with `train_ember.py` responsible for training the LightGBM-based classifier and `classify_binaries.py` handling inference on new or modified binaries.

Adversarial manipulation and PE modification functionality is isolated in the `test_files/` directory. Scripts such as `modify_pe.py`, `padding.py`, `codeCave.py`, and `perturbations.py` implement the perturbations. This boundary intentionally separates attack logic from detection logic, reducing the risk of entanglement between model training and adversarial generation. Generated adversarial samples are stored in the `generated_samples/` directory, reinforcing a clear data flow from original binaries to transformed artifacts.

Finally, analysis and interpretability artifacts are maintained inside the `test_files/` directory. SHAP-related outputs, including `shap_analysis.csv` and visualization images, are stored as standalone results rather than embedded into training scripts.

## References

- [1] Afshar, Bitra and Khorasani, Kash. *Briefing Notes*. URL: <https://www.concordia.ca/content/dam/ginacody/research/spnet/Documents/BriefingNotes/EmergingTech-MilitaryApp/BN-19-Emerging-technology-and-military-application-Oct2020.pdf>. (accessed: 12.11.2025).

- [2] Anderson, H. S. and Roth, P. "EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models". In: *ArXiv e-prints* (Apr. 2018). arXiv: 1804.04637 [cs.CR].
- [3] Anderson, Hyrum S et al. "Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning". In: *arXiv preprint arXiv:1801.08917* (Jan. 2018). arXiv: 1801.08917 [cs.CR].
- [4] Anderson, Hyrum S. et al. "Learning to Evade Static PE Machine Learning Malware Models via Reinforcement Learning". In: *Arxiv* (2018).
- [5] Aryal, Kshitiz, Gupta, Maanak, and Abdelsalam, Mahmoud. "Exploiting Windows PE Structure for Adversarial Malware Evasion Attacks". In: *ACM Digital Library* (2023).
- [6] Aryal, Kshitiz et al. "Explainability Guided Adversarial Evasion Attacks on Malware Detectors". In: *Arxiv* (2024).
- [7] Castro, Raphael Labaca, Schmitt, Corinna, and Rodosek, Gabi Dreo. "ARMED: How Automatic Malware Modifications Can Evade Static Detection?" In: *ResearchGate* (2019).
- [8] Costa, Joana C. et al. "How Deep Learning Sees the World: A Survey on Adversarial Attacks & Defenses". In: *IEEE Access* 12 (2024), pp. 61113–61136. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3395118. URL: <https://ieeexplore.ieee.org/document/10510296/> (visited on 05/30/2024).
- [9] DataScientest. *Adversarial Attack: Definition and protection against this threat*. URL: <https://datascientest.com/en/adversarial-attack-definition-and-protection-against-this-threat>. (accessed: 12.11.2025).
- [10] GeeksforGeeks. *Differences between Black Box Testing and White Box Testing*. URL: <https://www.geeksforgeeks.org/software-testing/differences-between-black-box-testing-vs-white-box-testing/>. (accessed: 12.11.2025).

- [11] GeeksforGeeks. *LightGBM (Light Gradient Boosting Machine)*. URL: <https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/>. (accessed: 12.11.2025).
- [12] GeeksforGeeks. *What is Feature Extraction?* URL: <https://www.geeksforgeeks.org/machine-learning/what-is-feature-extraction/>. (accessed: 12.11.2025).
- [13] Health, National Institutes of. *Dual-Use Research*. URL: <https://oir.nih.gov/sourcebook/ethical-conduct/special-research-considerations/dual-use-research>. (accessed: 12.11.2025).
- [14] Kaushik, Nupur, Bhardwaj, Vinay, and Arri, Harwant Singh. "A Machine Learning-Based Survey Of Adversarial Attacks And Defenses In Malware Classification". In: *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. 2023.
- [15] Kolosnjaji, Bojan et al. "Adversarial Malware Binaries: Evading Deep Learning for Malware Detection in Executables". In: *arxiv* (2018).
- [16] Luca Demetrio et al. "Functionality-preserving Black-box Optimization of Adversarial Windows Malware". In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 3469–3478. ISSN: 1556-6013, 1556-6021. DOI: 10.1109/TIFS.2021.3082330. arXiv: 2003.13526[cs]. URL: <http://arxiv.org/abs/2003.13526> (visited on 06/04/2024).
- [17] Lundberg, Scott and Lee, Su-In. "A Unified Approach to Interpreting Model Predictions". In: *Arxiv* (2017).
- [18] Nightfall. *Evasion Attacks*. URL: <https://www.nightfall.ai/ai-security-101/evasion-attacks>. (accessed: 12.11.2025).
- [19] Qi, Xuyan et al. "Adversarial Example Attacks Against Intelligent Malware Detection: A Survey". In: *2022 4th International Conference on Applied Machine Learning (ICAML)*. 2022 4th International Conference on Applied Machine Learning (ICAML). Changsha, China: IEEE, July 2022, pp. 1–7. ISBN: 978-1-66546-265-5. DOI: 10.1109/ICAML57167.2022.00068. URL: <https://ieeexplore.ieee.org/document/10056478/> (visited on 05/30/2024).
- [20] Smith, Emily. *Machine Learning for Malware Detection: Techniques, Models, and Industry Impact*. URL: <https://medium.com/@smith.emily2584/machine-learning-for-malware-detection-techniques-models-and-industry-impact-80f17472844e>. (accessed: 12.11.2025).
- [21] Wang, Zhibo et al. "Threats to Training: A Survey of Poisoning Attacks and Defenses on Machine Learning Systems". In: *ACM Computing Surveys* 55.7 (July 31, 2023), pp. 1–36. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3538707. URL: <https://dl.acm.org/doi/10.1145/3538707> (visited on 05/30/2024).